range are positive, that's the same as the indexes that correspond to the numbers in the range. But negative numbers correspond to indexes from the end of the string, so "abcdefg"[3...-1] returns "def", since d is the character at index 3, and f is the character at index -1, the last character. Halfway ranges also work, so "abcdefg"[...-1] returns all but the last character, and "abcdefg"[3...] returns everything from index 3 on. If both ends of the range are outside the string, the result is an empty string. If the ends of the range are in the wrong order relative to the string (like 4..2), you also get an empty string.

- Two arguments representing an index and a length, as in "abcdefg"[1, 3]. This gives you the substring starting at the given index, for the given length, in this case bcd. There are a few special cases. If the length goes off the end of the string, you just get the characters up to the end of the string—you don't get a bunch of whitespace padding or anything like that. If the length is zero or the index exactly matches the end of the string, you get an empty string. If the length is negative or if the index is greater than the length of the string, you get nil.
- The argument could be a regular expression, as in "abcdefg"[/d.*/]. In this case you get the first match, so the bracket syntax is equivalent to "abcedfg".match(/d.*/), but the brackets are way more confusing here. If there's no match, the return value is nil. A second argument is optional and it makes the expression return the capture group associated with the argument, as though you had returned a match object and looked up that value. If the second argument is a number that doesn't have a corresponding capture group, you get nil, if it's a name that doesn't have a corresponding named capture group, you get an IndexError.
- The argument can be a string as in "abcdefg"["def"]. In this case you get the string in the brackets if that substring exists in the receiving string, or you get nil. So, I guess equivalent to "abcdefg.include?("def") ? "def" : nil.

If you don't like the bracket syntax, the method is aliased as slice and takes the same arguments, so slice(1..3) or whatever. There's a destructive version—slice!—that returns the substring and modifies the original string by removing that substring:

```
sample = "abcdefg"
sample.slice!(1..3) # => "bcd"
sample              # => "aefg"
```

Other Substring Retrieval Methods

Some additional patterns of extracting substrings are common enough to have dedicated methods. The method strip returns a new version of the string with leading and trailing whitespace removed. The variant lstrip removes leading whitespace only, the variant rstrip removes trailing whitespace only. All three methods have ! variants that modify the string in place, so strip!, lstrip!, and rstrip!. The return values of all three modifying methods are the newly modified string if the string changed or nil if the string isn't changed. That's a common Ruby pattern for methods that mutate the original; however, if you are using the ! variants, you normally aren't using the return value, you're just using the receiving object later in the code.

For the purposes of the strip family of methods, Ruby defines whitespace as:

- A space character, unicode \x20 or " "
- A tab character, escaped as \t or unicode \x09
- A newline, or line feed, \n, or unicode \x0a
- A carriage return, \r or unicode \x0d